
CS 61B

Fall 2024

Midterm 2

Thursday, October 24, 2024

Your Name: _____

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 5 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	Total
Points:	15	18	32	23	12	100

- ☐ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible.**
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

Signature: _____

1 61Buff

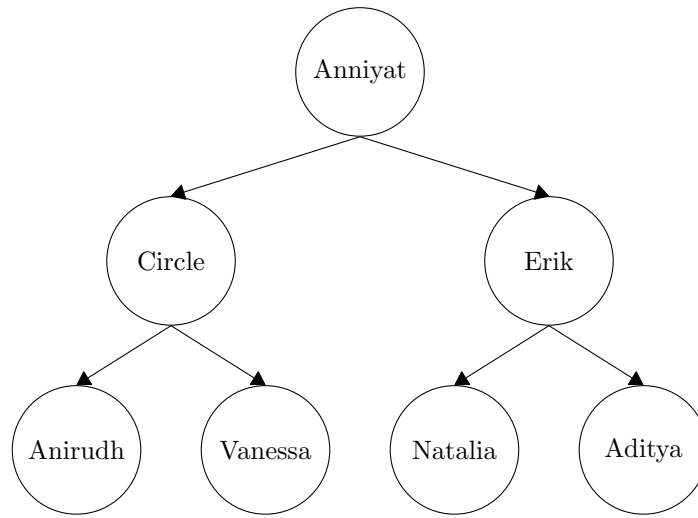
(15 points)

Aditya has organized a one-on-one arm wrestling competition amongst all of 61B's TAs. Aditya records the winner and loser of each battle and, in true 61B fashion, represents the leaderboard as a binary tree.

Each node represents a TA, and all nodes to the left of a node represent TAs who performed worse on the leaderboard, and all nodes to the right represent TAs who performed better on the leaderboard.

Below is the current leaderboard (in a list from first place to last) and the corresponding tree Aditya draws:

[Aditya, Erik, Natalia, Anniyat, Vanessa, Circle, Anirudh]



- (a) (1 point) Which of the following tree traversals gives us the leaderboard in order from best arm wrestler to worst?

☐ Pre-order
 ☐ In-order
 ☐ None of the above
 ☐ Post-order
 ☐ Level order

- (b) (1 point) Which of the following tree traversals gives us the reverse ordering of the leaderboard (worst to best)?

☐ Pre-order
 ☐ In-order
 ☐ None of the above
 ☐ Post-order
 ☐ Level order

- (c) (6 points) Consider a function that finds the leader of a (potentially unbalanced) leaderboard tree. What is the asymptotic best- and worst-case runtime of this function? Answer in terms of N , the total number of TAs in the tree.

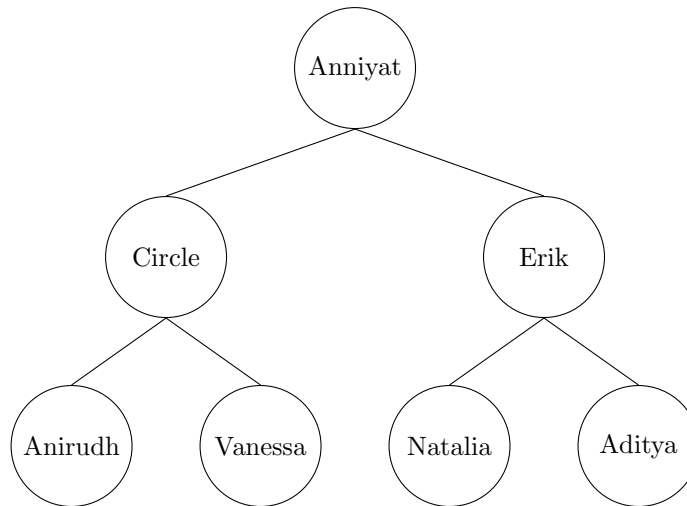
Best Case:

$\Theta(\quad)$

Worst Case:

$\Theta(\quad)$

The TAs don't like that Aditya has decided that he's the winner of his own competition. Thinking quickly, Aditya decides to claim that the diagram above is actually a min-heap (first place is towards the root).



- (d) (3 points) Each TA reads the heap, and decides to assume that their placement was the best possible placement that could have yielded the heap above. What is the best possible placement that each TA below could have according to the heap above?

Circle:	☐ 1	☐ 2	☐ 3	☐ 4	☐ 5	☐ 6	☐ 7
Anniyat:	☐ 1	☐ 2	☐ 3	☐ 4	☐ 5	☐ 6	☐ 7
Aditya:	☐ 1	☐ 2	☐ 3	☐ 4	☐ 5	☐ 6	☐ 7

Now the TAs are complaining that the leaderboard heap is ambiguous. Aditya decides to show the match results as a directed graph instead, where:

- Each vertex of the graph represents a TA.
- Each edge represents the outcome of a match, pointing from the winner to loser.
- Two TAs cannot face each other twice.

- (e) (2 points) If there are no cycles in this graph, then there is always an unambiguous first-place TA.

☐ True ☐ False

- (f) (2 points) If there are as many edges as there are vertices in the graph, then each TA must have faced every other TA in a match exactly once.

☐ True ☐ False

2 Disjoint Runtimes

(18 points)

In each subpart, determine the asymptotic runtime behavior. Answer in Θ notation, with respect to N .

For all Weighted Quick Union variants, tiebreak **connect** by connecting the lower-indexed root underneath the higher-indexed root.

(a) (6 points)

```
public static void f1(int N, QuickUnion qu) {
    Random rand = new Random();
    for (int i = 0; i < N; i += 1) {
        qu.connect(rand.nextInt(N), rand.nextInt(N));
    }
    for (int i = 0; i < N; i += 1) {
        if (!qu.isConnected(rand.nextInt(N), rand.nextInt(N))) {
            System.out.println("Dylan");
        }
    }
}
```

Best Case:

$\Theta(\quad)$

Worst Case:

$\Theta(\quad)$

(b) (6 points)

```
public static void f2(int N, WeightedQuickUnion wqu) {
    Random rand = new Random();
    for (int i = 0; i < N; i += 1) {
        wqu.connect(rand.nextInt(N), rand.nextInt(N));
    }
    for (int i = 0; i < N; i += 1) {
        if (!wqu.isConnected(rand.nextInt(N), rand.nextInt(N))) {
            System.out.println("Gaston");
        }
    }
}
```

Best Case:

$\Theta(\quad)$

Worst Case:

$\Theta(\quad)$

- (c) (6 points) As a reminder, the `isConnected` and `connect` methods of `WeightedQuickUnionPathCompression` each run in $\Theta(\alpha(N))$ amortized time.

```
public static void f3(int N, WeightedQuickUnionPathCompression wqu) {
    Random rand = new Random();
    for (int i = 0; i < N; i += 1) {
        wqu.connect(rand.nextInt(N), rand.nextInt(N));
    }
    for (int i = 0; i < N; i += 1) {
        if (!wqu.isConnected(rand.nextInt(N), rand.nextInt(N))) {
            System.out.println("Hamuy");
        }
    }
}
```

Best Case:

$\Theta(\quad)$

Worst Case:

$\Theta(\quad)$

This page is intentionally left (mostly) blank.

3 Unstable Matching Algorithm

(32 points)

In preparation for Project 3, Anniyat decides to devise a new partner-matching scheme.

In order to design our scheme, we first make a `StrangeHashSet<E>` which behaves as a `HashSet`, except for the following ways:

1. The `StrangeHashSet` starts with **one** bucket.
 2. The `StrangeHashSet` resizes when **any bucket contains 3 or more elements**. This can trigger multiple resizes in a single `add` operation.
 3. When the `StrangeHashSet` resizes, the number of buckets is **doubled**.
 4. The `StrangeHashSet` has a method `public List<E>[] getBuckets()` which returns an array of Lists. The i th List contains the values stored in the i th bucket of the `StrangeHashSet` (in any order).
- (a) (3 points) We create a new `StrangeHashSet<Integer>` and insert the following elements in the given order:

[5, 6, 1, 10, 13, 4]

What is the result of calling `getBuckets()`? In each box, write a list like [5, 4], or write [] if the bucket is empty.

Note: The `hashCode` of an `Integer` is itself.

0	→	
1	→	
2	→	
3	→	
4	→	
5	→	
6	→	
7	→	

To set up our matching algorithm, each student is asked to submit a **unique** password. These are then stored in a (regular) `Set<String>` passwords.

We then run the following matchmaking algorithm:

```
public class Matching {
    /**
     * Pairs the student with password a, and the student with password b.
     * If a is paired with b, then b is also paired with a.
     */
    private static void pair(String a, String b) { ... } // Implementation omitted.

    /**
     * Pairs all students according to their passwords.
     * If there are an odd number of students, pairs all but one student,
     * and returns the password of the unpaired student.
     * If there are an even number of students, pairs all students, and returns null.
     */
    public static String pairAllStudents(Set<String> passwords) {
        while (passwords.size() > 1) {
            // 1: Create a new StrangeHashSet.
            StrangeHashSet<String> matcher = new StrangeHashSet<>();

            // 2: Insert all passwords into the StrangeHashSet.
            for (String s : passwords) {
                matcher.add(s);
            }

            // 3: If any bucket contains exactly two passwords, pair the corresponding students.
            //      Then remove both passwords from passwords.
            for (List<String> bkt : matcher.getBuckets()) {
                if (bkt.size() == 2) {
                    pair(bkt.get(0), bkt.get(1));
                    passwords.remove(bkt.get(0));
                    passwords.remove(bkt.get(1));
                }
            }
        } // 4: Repeat Steps 1-3 until there are fewer than two students unpaired.

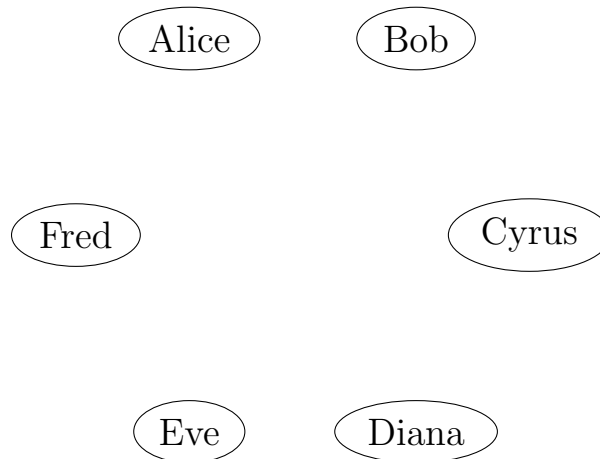
        // 5: If there's one student unpaired, return the password of that student.
        if (passwords.size() == 1) {
            return passwords.toArray()[0];
        }

        // 6: Otherwise return null.
        return null;
    }
}
```

(b) (3 points) We run `pairAllStudents` on the following students:

- Alice submits a password whose hash is 5.
- Bob submits a password whose hash is 6.
- Cyrus submits a password whose hash is 1.
- Diana submits a password whose hash is 10.
- Eve submits a password whose hash is 13.
- Fred submits a password whose hash is 4.

What are the resulting pairs? Draw 3 lines to match the students into pairs.



For the rest of the question, assume that all passwords have distinct hash codes, unless otherwise stated.

(c) (12 points) Which of the following statements are true? Select all that apply.

- ☐ Assume that hash codes range from 0 to $2^b - 1$ for some integer b . When inserting all students into a single `StrangeHashSet`, at most b resizes occur.
- ☐ If a bucket contains three items immediately before a resize, it is possible for the items to end up in three different buckets after the resize.
- ☐ In each iteration of the `while` loop, at least one pair of students will be created.
- ☐ In each iteration of the `while` loop, at least half of the remaining students will be paired.
- ☐ If an iteration of the `while` loop requires k resizes when adding all students to the `StrangeHashSet`, then the next iteration of the `while` loop will require strictly fewer than k resizes.
- ☐ `pairAllStudents` is guaranteed to terminate.
- ☐ None of the above

- (d) (2 points) Mia and Sebastian want to form a partnership, and decide to coordinate their passwords so they end up partnered (with high probability).

First, Mia selects a password whose hash is 0. What should the hash of Sebastian's password be in order to maximize the chances that he is paired with Mia? Assume that:

- Hash codes range from 0 to $2^{32} - 1$.
- Students may not pick passwords with the same hash code.

- (e) (12 points) Consider the following modifications to `pairAllStudents`. For each modification, find a list of at most five password hashes that would cause `pairAllStudents` to infinite-loop.

Each modification is independent. If no list of password hashes can cause an infinite loop, write "Impossible".

As an example, you can write `[0, 2]` to represent a list containing one password with hash 0, and one password with hash 2.

1. Instead of **doubling** the number of buckets on a resize, we **triple** the number of buckets.

2. Two passwords are allowed to share a hash code, but no three passwords share the same hash code.

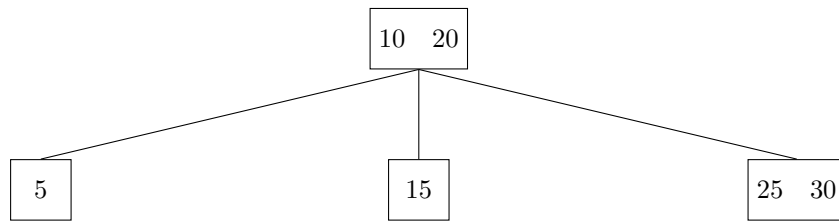
3. Exactly one hash code is shared by three passwords, and all other passwords have unique hash codes.

4. The `StrangeHashSet` begins with **two** buckets, instead of **one** bucket.

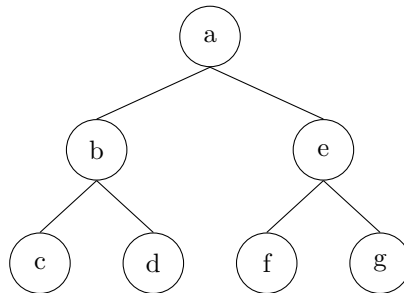
4 Fix LLRB

(23 points)

(a) (9 points) Convert the 2-3 tree below into its corresponding Left-Leaning Red-Black Tree (LLRB).



Select the value that goes in each node of the LLRB below. Not all nodes need to be used; select “None” if the node is unused.



- | | | | | | | | |
|----|-------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|----------------------------|
| a: | ☐ 5 | ☐ 10 | ☐ 15 | ☐ 20 | ☐ 25 | ☐ 30 | ☐ None |
| b: | ☐ 5 | ☐ 10 | ☐ 15 | ☐ 20 | ☐ 25 | ☐ 30 | ☐ None |
| c: | ☐ 5 | ☐ 10 | ☐ 15 | ☐ 20 | ☐ 25 | ☐ 30 | ☐ None |
| d: | ☐ 5 | ☐ 10 | ☐ 15 | ☐ 20 | ☐ 25 | ☐ 30 | ☐ None |
| e: | ☐ 5 | ☐ 10 | ☐ 15 | ☐ 20 | ☐ 25 | ☐ 30 | ☐ None |
| f: | ☐ 5 | ☐ 10 | ☐ 15 | ☐ 20 | ☐ 25 | ☐ 30 | ☐ None |
| g: | ☐ 5 | ☐ 10 | ☐ 15 | ☐ 20 | ☐ 25 | ☐ 30 | ☐ None |

Select all red nodes:

- ☐ a
 ☐ b
 ☐ c
 ☐ d
 ☐ e
 ☐ f
 ☐ g

- (b) (14 points) Write `convert`, a function that takes in a 2-3 tree and converts it to its corresponding LLRB.

A 2-3 tree node is defined as follows:

```
public class TwoThreeTreeNode {
    public Integer[] keys;
    public TwoThreeTreeNode[] children;

    // Creates a 2-node
    public TwoThreeTreeNode(Integer key) {
        this.keys = new Integer[]{key, null};
        this.children = new TwoThreeTreeNode[]{null, null, null}; // Ordered from left to right
    }
}
```

Assumptions:

- In a 2-node, `keys[1]` and `children[2]` are null.
- In a 3-node, `keys[0]` and `keys[1]` are not null, and `keys[0] < keys[1]`.

```
public class LLRB {
    public LLRBNode root;

    public class LLRBNode {
        public int key;
        public LLRBNode left, right;
        public boolean isRed;

        public LLRBNode(int key, boolean isRed) {
            this.key = key;
            this.isRed = isRed;
        }
    }

    public LLRB(TwoThreeTreeNode node) {
        root = convert(node);
    }
}
```

```

private LLRBNode convert(TwoThreeTreeNode root) {

    if (root == null) {
        return null;
    }

    if ( _____ ) {

        LLRBNode node = _____;

        _____ = _____;

        _____ = _____;

        return node;
    }

    LLRBNode leftChild = _____;

    LLRBNode parent = _____;

    parent.left = leftChild;

    _____ = _____;

    _____ = _____;

    _____ = _____;

    return parent;
}

```

5 Yokeyrin's Cave

(12 points)

Anirudh the Adventurer is on a quest to obtain the 61B Midterm 2 answer key. He ventures into Mastermind Yokeyrin's cave with 100 rooms, each numbered 1 to 100. He starts in Room 1, the cave's entrance, and he must reach Room 100 in as little time as possible to obtain the answer key!

From each room, Anirudh can move to any adjacent room in one minute. Fill out the `shortestTime` method, which is defined as follows:

- Input: `Map<Integer, Set<Integer>> adj` which maps each room (1–100) to a set of all adjacent rooms.
- Output: The minimum number of minutes needed to go from Room 1 to Room 100. If it is impossible to reach Room 100, return `-1`.

```
public class Queue<T> {

    /** Creates a new queue */
    public Queue() { ... }

    /** Adds item to the back of the queue */
    public void add(T item) { ... }

    /** Removes and returns the front item of the queue */
    public T remove() { ... }

    /** Returns true if the queue has no elements */
    public boolean isEmpty() { ... }
}

public class CaveStory {
    private static class State {
        public int room;
        public int mins;

        public State(int room, int mins) {
            this.room = room;
            this.mins = mins;
        }
    }
}
```

```

public static int shortestTime (Map<Integer, Set<Integer>> adj) {
    Set<Integer> traversed = new HashSet<>();
    Queue<State> options = new Queue<>();

    options.add(new State(1, 0));

    while (!options.isEmpty()) {

        State curr = options.remove();

        if (_____1_____) {
            _____2_____;
        }

        if (_____3_____) {
            _____4_____;

            for (_____5_____ : _____6_____) {
                _____7_____;
            }
        }
    }
    return -1; // No path exists
}

```

Nothing on this page is worth any points.

Bonus Question

(0 Points)

For the following modification to `pairAllStudents` from Question 3, find a list of at most five password hashes that would cause `pairAllStudents` to infinite-loop. If no set of password hashes can cause an infinite loop, write “Impossible”:

The `StrangeHashSet` resizes when the **average number of elements in a bucket is strictly greater than two**. After inserting all items into the `StrangeHashSet`, if a bucket contains three or more elements, recursively pair students within that bucket using `pairAllStudents`, leaving a single student unpaired if the bucket contained an odd number of students.

Feedback

(0 Points)

Leave any feedback, comments, concerns, or more drawings below!